“Smart Product Search & Recommendation System”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Venkatesh Addanki	**:2520030404**
Pranav Chowdary	**:2520030555**
Laxman Reddy	**:2520030097**

Team Number: 4

Section:

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Smart Product Search & Recommendation System"** is a record of Bonafide work of **Pranav Chowdary – 2520030555, Venkatesh Addanki – 2520030404, Laxman Reddy– 2520030097** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Venkatesh Addanki – 2520030404

Pranav Chowdary – 2520030555

Laxman Reddy – 2520030097

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Smart Product Search & Recommendation System"** is a bonafide work done and submitted by **VENKATESH ADDANKI – 2520030404, PRANAV CHOWDARY – 2520030555, LAXMAN REDDY – 2520030097** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to our guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his constant encouragement, valuable suggestions and technical guidance throughout the development of our project “Smart Product Search & Recommendation System”. His clear explanations of advanced data structures and algorithms helped us choose the right algorithm for each part of the system.

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the laboratory facilities and a supportive academic environment. We also thank all the faculty members of the department and the management of Koneru Lakshmaiah Education Foundation for their support.

Finally, we thank our friends and family for their motivation and cooperation, which helped us complete this project successfully.

Venkatesh Addanki – 2520030404

Pranav Chowdary – 2520030555

Laxman Reddy – 2520030097

Abstract

Online shopping platforms list thousands of products, so finding the right item quickly is difficult. Users make spelling mistakes, type only partial product names and have to compare prices across several stores by hand. Traditional keyword matching often returns irrelevant results and becomes slow as the product catalogue grows. This project, the Smart Product Search & Recommendation System, applies advanced data structures and algorithms to make product discovery fast, accurate and easy to compare.

The system is built with a Java back end that exposes REST APIs using the built-in HTTP server, and a web front end written in HTML, CSS and JavaScript. It stores a catalogue of 24 products in 9 categories with Amazon and Flipkart prices. Product search is performed with the Knuth-Morris-Pratt (KMP) algorithm, and results can be sorted by price (Quick Sort), name (Merge Sort) or rating (Heap Sort). Users can also add products to a shortlist and compare store prices side by side. An Advanced Algorithms Lab implements the six course outcomes: an algorithm advisor (CO1), string matching with KMP, Z-Function, Rabin-Karp and Suffix Array (CO2), a bundle optimiser using bitmask, subset, interval and tree dynamic programming (CO3), platform assignment with Ford-Fulkerson, Edmonds-Karp and Dinic network-flow algorithms (CO4), exact versus greedy approximation for NP-hard selection (CO5), and randomized and parallel algorithms (CO6).

Each module runs on real product data and reports the measured execution time, so that the practical behaviour of the algorithms can be compared with their theoretical complexity. The project demonstrates how classical algorithms solve real e-commerce problems such as fast retrieval, price comparison and budget-based product selection, and it can be extended with spelling correction (Trie and Levenshtein distance) and a database back end.

Table of Contents

			Page No.
	Acknowledgement		4
	Abstract		5
	Table of Contents		6
	List of Figures		7
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	8-10
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements 2.3 Hardware and Software Requirements	11-12
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer 3.3 Data structures and algorithms used	13-15
4	Implementation	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 Build and execution	16-18
5	Results	5.1 User interface results 5.2 Algorithm lab results 5.3 Performance observations	19-26
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	27-28
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	29-30
	References	- Books, research papers and documentation used	31

List of Figures

1.1 Overall architecture of the Smart Product Search system 8

3.1 Module and algorithm map (CO1 to CO6) 13

5.1 Home page with search bar and category tiles 19

5.2 Search results for “shoe” sorted by price 20

5.3 Product comparison shortlist 20

5.4 Algorithm Advisor (CO1) 21

5.5 Advanced search: KMP, Z-Function, Rabin-Karp and Suffix Array (CO2) 21

5.6 Smart Bundle Optimizer using bitmask DP (CO3) 22

5.7 Platform assignment network flow (CO4) 23

5.8 NP / NP-Hard optimisation lab (CO5) 24

5.9 Randomized algorithms: Las Vegas and Monte Carlo (CO6) 24

5.10 Parallel processing lab and work-span analysis (CO6) 25

1. INTRODUCTION

1.1 Overview

Online shopping has grown rapidly, and platforms such as Amazon and Flipkart now list millions of products. For a shopper, the real difficulty is not the lack of choice but finding the right product quickly and at the best price. The Smart Product Search & Recommendation System is a web-based application that makes product discovery faster and smarter by using advanced data structures and algorithms studied in the course Data Structures and Algorithms-3 (25CS2103E).

The system has three parts, shown in Fig. 1.1. A browser-based front end lets the user search, filter, sort and compare products. A Java back end exposes REST APIs through the built-in HTTP server and processes every request. A data layer holds a catalogue of 24 products in 9 categories (Mobiles, Laptops, Headphones, Shoes, Fashion, Groceries, Personal Care, Home and Electronics) with Amazon and Flipkart prices. The back end uses KMP for keyword search and Quick Sort, Merge Sort and Heap Sort for ordering the results by price, name and rating.

In addition to the shopping features, the project contains an Advanced Algorithms Lab that covers the six course outcomes (CO1 to CO6). It lets the user run string-matching algorithms, dynamic-programming bundle optimisation, network-flow platform assignment, exact and approximate NP-hard optimisation, and randomized and parallel algorithms on the same product data, and it shows the execution time measured by the Java back end.

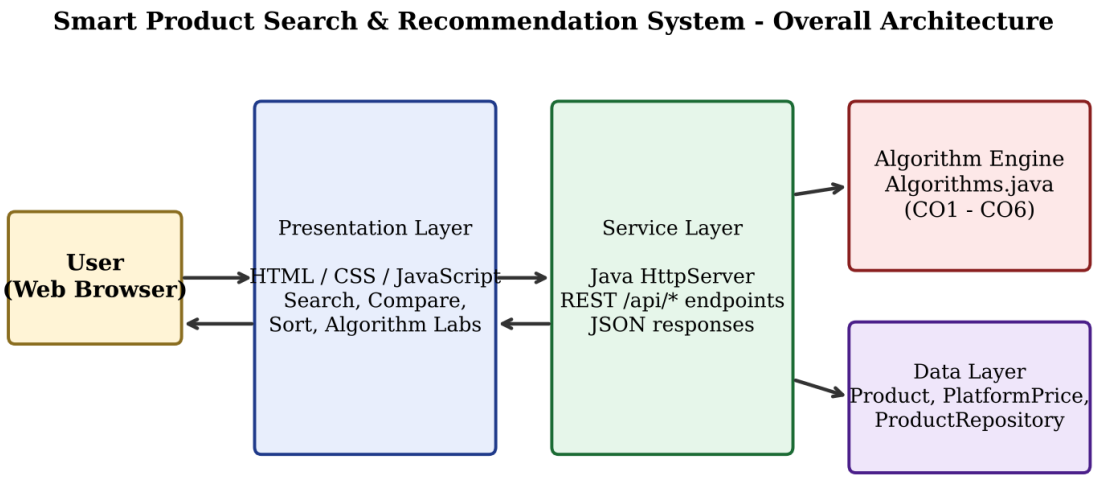


Fig. 1.1 Overall architecture of the Smart Product Search system

1.2 Problem Statement

Product catalogues are large and users search in imperfect ways. The main problems addressed by

this project are:

- Users frequently make spelling mistakes and type incomplete or partial product names.
- Simple keyword matching returns irrelevant results and does not scale well as the number of products grows.
- Prices of the same product differ between platforms, and comparing them manually is slow.
- Choosing a set of products within a fixed budget is a combinatorial problem that is hard to solve by hand.
- Discovering similar or alternative products is difficult without an organised, sorted and categorised view of the data.

The problem is therefore to design and implement a system that retrieves products quickly, orders them by meaningful criteria, compares prices across platforms and applies suitable algorithms to the harder optimisation tasks, while clearly showing how each algorithm behaves.

1.3 Scope of the Project

The scope of the project covers the following areas:

- A product catalogue of 24 products in 9 categories with example Amazon and Flipkart prices, ratings and descriptions.
- Keyword and partial-name search with category filtering, and sorting by price, name and rating.
- A shortlist of up to four products with side-by-side store prices and links to the store search pages.
- Seven interactive algorithm modules for CO1 to CO6 running on the catalogue through REST APIs.
- Twelve groups of classical algorithms (searching, sorting, trees, graph traversal, shortest path, minimum spanning tree, dynamic programming, hashing and greedy) implemented in Java.

The prices in the catalogue are reference values stored in the program. Live price fetching from the shopping websites, user accounts and payment are outside the scope of this project.

1.4 Objectives

- To retrieve products quickly and efficiently using string-matching algorithms.
- To sort products by price, name and rating using Quick Sort, Merge Sort and Heap Sort.

- To compare prices across platforms and highlight the best listed price.
- To select the best combination of products within a budget using dynamic programming.
- To model product-to-platform assignment as a maximum-flow problem and compare Ford-Fulkerson, Edmonds-Karp and Dinic.
- To compare exact and approximate solutions for an NP-hard selection problem.
- To demonstrate randomized and parallel algorithms on product price data and measure their running time.
- To demonstrate the practical use of advanced data structures and algorithms in solving real-world e-commerce problems.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

The functional requirements describe what the system must do. They are listed in Table 2.1.

Table 2.1 Functional requirements

ID	Requirement	Module / API
FR1	Display the product catalogue with name, category, rating, minimum price and store prices.	/api/products
FR2	Search products by keyword or partial name (and category) using KMP.	/api/search
FR3	Filter products by category and sort by price, name or rating.	/api/search, /api/categories
FR4	Add up to four products to a shortlist and compare store prices side by side.	/api/compare
FR5	Recommend an algorithm family for a given problem type with its time and space complexity.	/api/advisor (CO1)
FR6	Search the product corpus with KMP, Z-Function, Rabin-Karp or Suffix Array and report matches and time.	/api/advanced-search (CO2)
FR7	Find the best product bundle within a user-given budget using dynamic programming.	/api/bundle (CO3)
FR8	Assign products to platforms using maximum flow and report the flow and time.	/api/flow (CO4)
FR9	Compare exact and greedy approximate selection for a budget and report the approximation ratio.	/api/np-optimize (CO5)
FR10	Run randomized (Las Vegas, Monte Carlo) and parallel (prefix sum, reduce) algorithms on price data.	/api/randomized, /api/parallel (CO6)

2.2 Non-Functional Requirements

- Performance: search and sorting over the catalogue should respond in a few milliseconds; each algorithm module reports its measured time.
- Usability: a single-page web interface that works in a normal browser without installation of any plug-in, with clear tables and cards.
- Reliability: the algorithms always return a correct answer for valid input; invalid input such as an empty query or unknown product returns an error message instead of failing.
- Portability: the back end uses only the standard Java library, so it runs on any system with a JDK; the front end uses plain HTML, CSS and JavaScript.
- Maintainability: algorithms are kept in one class (Algorithms.java) separate from the HTTP

handlers (Main.java) and the data (Product.java, ProductRepository.java).

- Scalability: the algorithm complexity (for example O(n+m) for KMP and O(n log n) for sorting) allows the same code to work for larger catalogues.

2.3 Hardware and Software Requirements

Table 2.2 Hardware and software requirements

Item	Requirement
Processor	Intel Core i3 or equivalent (or higher)
RAM	4 GB minimum (8 GB recommended)
Storage	100 MB free disk space
Operating system	Windows 10/11, Linux or macOS
Programming language	Java (JDK 11 or later), JavaScript (ES6)
Front-end technologies	HTML5, CSS3, JavaScript Fetch API
Back-end technology	Java built-in HttpServer (com.sun.net.httpserver)
Tools	Git and GitHub, VS Code / IntelliJ IDEA, web browser (Chrome or Edge)

3. SYSTEM DESIGN

3.1 High-level Architecture Diagram

The system follows a layered client–server design, as shown in Fig. 1.1 and Fig. 3.1. The user interacts with the web page in the browser. The page calls the REST APIs of the Java server using the Fetch API and receives JSON responses. Each API handler in Main.java calls the required methods of the Algorithms class and builds the JSON response from the Product objects of the repository.

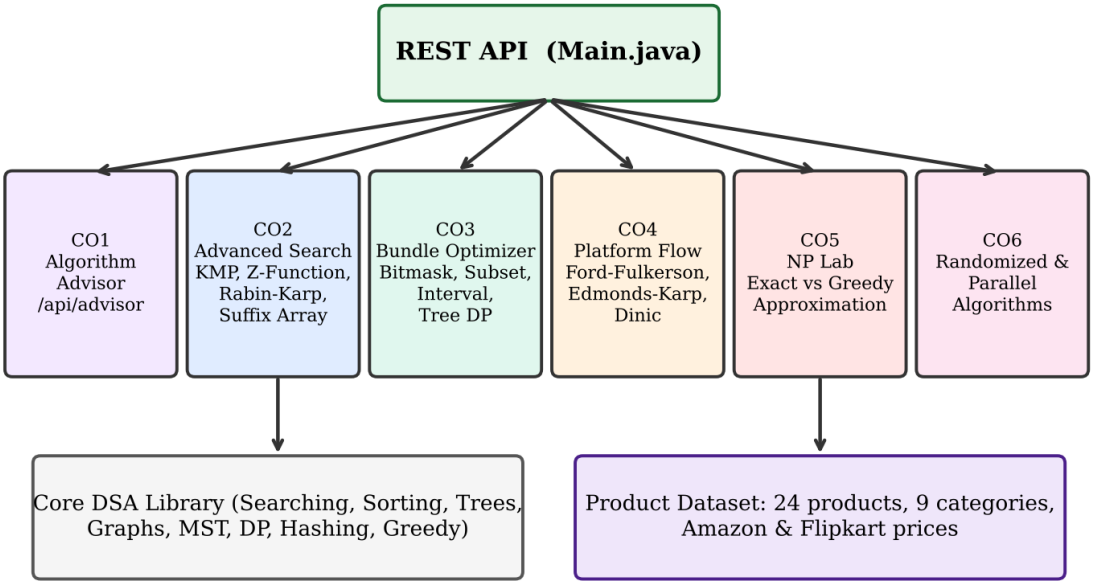


Fig. 3.1 Module and algorithm map (CO1 to CO6)

3.2 Description of each layer

Presentation layer. The front end consists of index.html, styles.css and app.js. It renders the product cards, category filters, the shortlist table and the seven algorithm modules. State such as the selected category, the search text and the shortlist is kept in JavaScript variables, and the page is updated without reloading.

Service layer. Main.java starts an HttpServer on port 8080 and registers the contexts /api/products, /api/search, /api/compare, /api/categories, /api/algorithm-status, /api/advanced-search, /api/bundle, /api/flow, /api/np-optimize, /api/randomized, /api/parallel and /api/advisor. The root context serves the static front-end files. Cross-origin headers are added to every response.

Algorithm layer. Algorithms.java contains the implementation of all algorithms: searching (KMP, linear search, Z-Function, Rabin-Karp, suffix array with LCP), sorting (bubble, selection, insertion,

merge, quick, heap, counting and radix), trees (BST and AVL), graph algorithms (BFS, DFS, Dijkstra, Bellman-Ford, Prim, Kruskal), dynamic programming (Floyd-Warshall, LCS, bitmask, subset, interval and tree DP), network flow, NP and approximation methods, randomized and parallel algorithms, hashing and greedy methods. Separate folders in the repository (01_Searching to 12_Greedy) keep a standalone copy of two algorithms per topic.

Data layer. Product.java stores the id, name, category, rating, description and a list of PlatformPrice objects. ProductRepository.java seeds the catalogue with 24 products and generates the store search links for Amazon and Flipkart. The minimum price of a product is computed over the supported platforms.

3.3 Data structures and algorithms used

Table 3.1 lists the main data structures and algorithms of the system, their purpose and complexity, together with the course outcome that they satisfy.

Table 3.1 Algorithms, purpose and course outcome mapping

CO	Algorithm / Data structure	Purpose in the project	Time complexity
CO1	Algorithm Advisor (problem classification)	Choose the right algorithm family for text search, bundle, assignment, NP-hard and large-data problems	–
CO2	KMP	Keyword search in product names and categories	O(n + m)
CO2	Z-Function, Rabin-Karp, Suffix Array + LCP	Alternative pattern matching on name, category and description	O(n + m); O(m log n) query for suffix array
CO3	Bitmask, Subset (0-1 Knapsack), Interval and Tree DP	Select products within a budget to maximise rating score	$O(2^n \cdot n)$; O(n·W); $O(n^3)$; O(n·W)
CO4	Ford-Fulkerson, Edmonds-Karp, Dinic	Assign products to platforms with capacity constraints	O(E·f); $O(VE^2)$; $O(V^2E)$
CO5	Exact brute force and greedy approximation	Compare optimal and approximate subset selection	$O(2^n)$; O(n log n)
CO6	Las Vegas QuickSelect, Miller-Rabin (Monte Carlo)	Randomized k-th smallest price and primality testing	O(n) expected; O(k $\log^2$ n)
CO6	Parallel prefix sum and parallel reduce (ForkJoinPool)	Parallel processing of price arrays with work-span analysis	O(n) work, O(log n) span
–	Quick Sort, Merge Sort, Heap Sort	Sort results by price, name and rating	O(n log n)
–	HashMap, Separate Chaining	Fast product lookup by name	O(1) average

CO	Algorithm / Data structure	Purpose in the project	Time complexity
–	BFS, DFS, Dijkstra, Bellman-Ford, Prim, Kruskal, Floyd-Warshall, LCS	Demonstration on the platform graph and product names	As per algorithm

4. IMPLEMENTATION

4.1 Module-wise implementation

Product search and sorting. The /api/search handler reads the query text, the category and the sort option. For every product of the selected category it runs KMP on the lower-case product name and on the category. The matching products are then ordered: Quick Sort on the minimum price for "price", Merge Sort on the name for "name", and Heap Sort (a max priority queue) on the rating for "rating". The KMP failure function (LPS array) is built in O(m) and the text is scanned in O(n) without re-comparing characters.

Price comparison. The /api/compare handler finds a product by name, keeps only the supported platforms (Amazon and Flipkart), sorts the prices in ascending order and returns the best price with the platform list and store links.

CO1 – Algorithm Advisor. For five problem types (text search, bundle optimisation, platform assignment, NP-hard optimisation and large dataset processing) the /api/advisor handler returns the problem class and the suitable algorithms with the reason for choosing each of them and their time and space complexity.

CO2 – Advanced search. A corpus string is created for every product by joining its name, category and description in lower case. The pattern is searched with the selected algorithm: KMP (kmpCount), Z-Function (zSearch on pattern + "$" + text), Rabin-Karp (rolling hash with base 31 and modulus 10^9+7, with a string comparison on every hash match) or Suffix Array (prefix-doubling construction in $O(n \log^2 n)$, Kasai's LCP array and a binary search for the left and right boundaries). The handler returns the number of matching products, the total occurrences and the time in milliseconds. The "Compare All" option runs all four algorithms for the same query.

CO3 – Bundle optimiser. The user enters a budget and optional categories. Prices are taken as the minimum store price and the score of a product is its rating multiplied by 100. Bitmask DP checks all 2^n subsets (n up to 20) and keeps the feasible subset with the highest total score. Subset DP is the 0-1 knapsack table dp[i][w]; interval DP fills dp[i][j] over product intervals; tree DP works on a category tree. The result shows the selected products, total cost, remaining budget and execution time.

CO4 – Platform assignment by network flow. A flow network is built with a source, six products, three platforms and a sink. Source-to-product edges and platform-to-sink edges have capacities, and a product-to-platform edge exists when the platform lists the product. Ford-Fulkerson (DFS augmenting paths), Edmonds-Karp (BFS augmenting paths) and Dinic (level graph with blocking

flow) compute the maximum flow, and the minimum cut is found from the residual graph. The front end draws the network with the used edges highlighted.

CO5 – NP lab. The selection of products within a budget is the 0-1 knapsack / subset-sum type of NP-hard problem. The exact method enumerates all 2^n subsets, and the greedy method sorts products by score-to-price ratio and picks them while the budget allows. The handler reports the score, cost and time of both methods and the approximation ratio (greedy score divided by exact score). The page also explains the classes P, NP, NP-Complete and NP-Hard and the reduction chain 3-SAT $\leq_p$ Vertex Cover $\leq_p$ Independent Set.

CO6 – Randomized and parallel algorithms. The Las Vegas algorithm is randomized QuickSelect for the k-th smallest product price: it is always correct and only its running time depends on the random pivots. The Monte Carlo algorithm is the Miller-Rabin primality test with k random witnesses, whose error probability is at most 4^{-k}. The parallel module computes a prefix sum and a reduce (sum) of the price array with Java ForkJoinPool and compares them with the sequential versions. It also reports work, span and parallelism (work divided by span).

Other algorithm modules. The /api/algorithm-status handler runs the remaining algorithms on live data: BFS and DFS on a graph of six shopping platforms, Dijkstra and Bellman-Ford for the shortest distance from Amazon, Floyd-Warshall on a small distance matrix, LCS of "iphone 15" and "iphone 16", a hash bucket for a product name, activity selection and fractional knapsack.

4.2 Front-end logic (UI rendering, state management)

The front end is a single page built with plain HTML, CSS and JavaScript. When the page loads, the init() function fetches /api/categories and /api/products and renders the category tiles, the filter list and the product cards. The functions setCategory(), syncSearch() and loadProducts() keep the search box, category and sort option in sync and request /api/search with the current values.

Each product card shows the rating, the best listed price, the store icons and two buttons: "Add to compare" and "View deal". The shortlist is stored in a JavaScript set (maximum four products), and renderCompare() builds a table with the best price and the Amazon and Flipkart prices of each selected product. The algorithm modules are switched by showModule(); each module has a run function (for example runAdvancedSearch(), runBundleOptimizer(), runFlowAlgo(), runNpLab(), runRandomized() and runParallel()) that calls the corresponding API and a render function that shows the metrics and tables. The flow network of CO4 is drawn as an SVG graph.

4.3 API integration

The front end communicates with the back end only through HTTP GET requests that return JSON. A small helper function api(url) wraps fetch() and throws an error for a failed response. Table 4.1 lists the endpoints.

Table 4.1 REST API endpoints of the back end

Endpoint	Parameters	Purpose
/api/products	–	All products with prices and store links
/api/categories	–	List of categories
/api/search	q, category, sort	KMP search with category filter and sorting
/api/compare	product	Sorted store prices and best price
/api/algorithm-status	–	Live results of the basic algorithm library
/api/advisor	problem	Algorithm recommendation (CO1)
/api/advanced-search	q, algo	KMP, Z, Rabin-Karp, Suffix Array search (CO2)
/api/bundle	budget, algo, categories	Bundle optimisation by DP (CO3)
/api/flow	algo	Maximum-flow platform assignment (CO4)
/api/np-optimize	budget, n	Exact versus greedy selection (CO5)
/api/randomized	k, num, witnesses, seed	Las Vegas and Monte Carlo (CO6)
/api/parallel	–	Parallel prefix sum and reduce (CO6)

4.4 Build and execution

The project is built with the standard JDK. The script run.bat compiles all files of backend/src into backend/bin with javac and then starts the server with “java -cp bin Main” from the backend folder. The application is opened in a browser at http://localhost:8080. The source code is maintained in a Git repository (KLH-CSE-2026-2027-TEAM4-SmartProductSearch) by the three team members. The folder legacy contains the first console version of the project, in which products and prices were stored in a HashMap and searched from a text menu, and the folder algorithms contains the stand-alone algorithm programs grouped by topic.

5. RESULTS

5.1 User interface results

Fig. 5.1 shows the home page with the search bar and the category tiles. Fig. 5.2 shows the result of searching “shoe” with the sort option “Price: low to high”: the three matching products (Puma Running Shoes ₹3,799, Adidas Grand Court ₹4,699 and Nike Air Max ₹7,999) appear in ascending order of price, produced by KMP and Quick Sort. Fig. 5.3 shows three products added to the shortlist and their Amazon and Flipkart prices side by side.

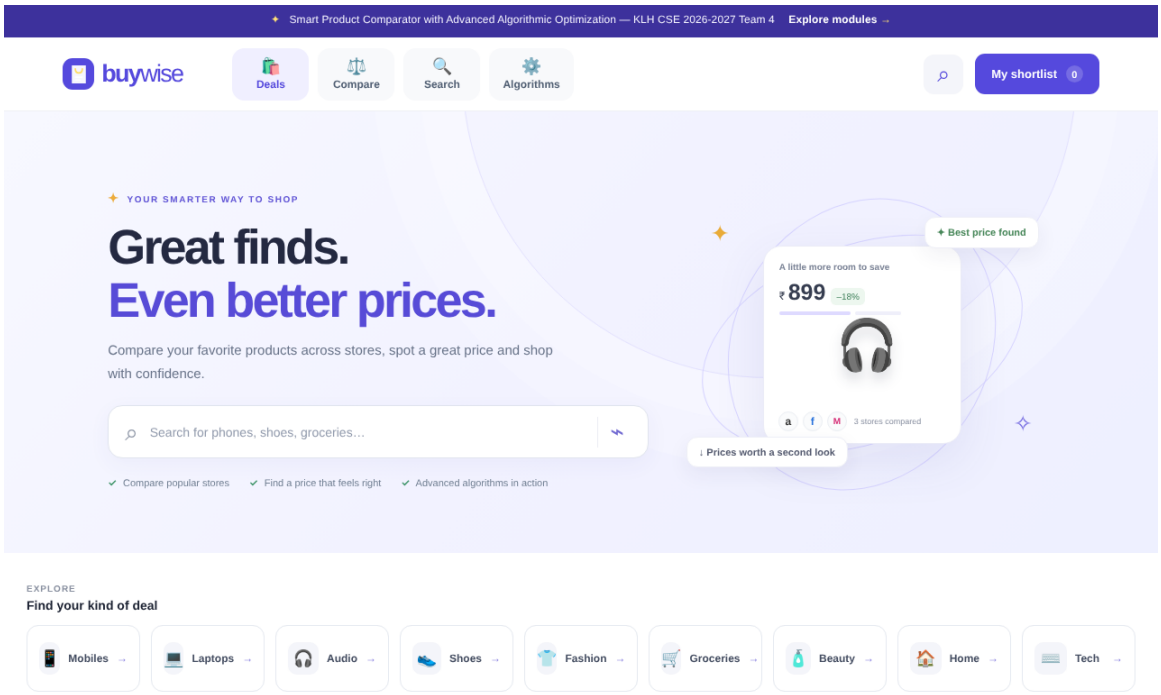


Fig. 5.1 Home page with search bar and category tiles

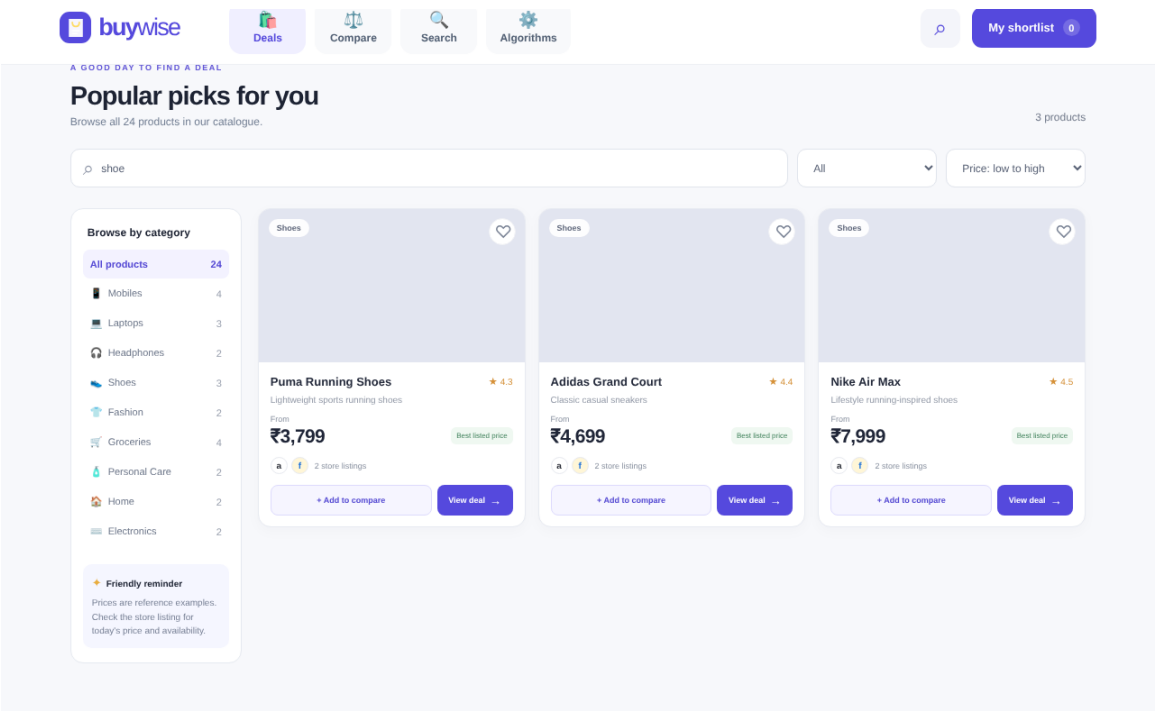


Fig. 5.2 Search results for “shoe” sorted by price

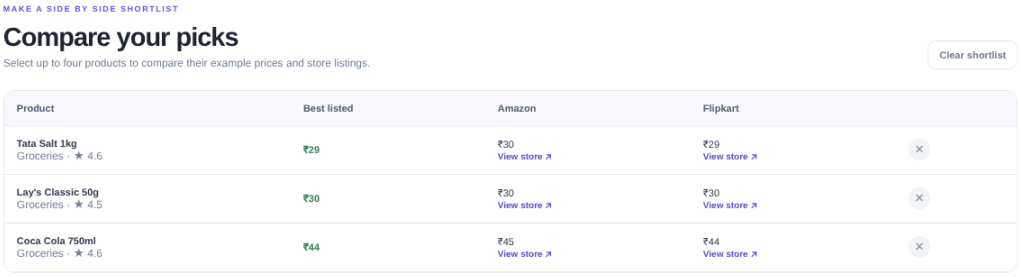


Fig. 5.3 Product comparison shortlist

5.2 Algorithm lab results

CO1 – Algorithm Advisor. For the problem “Product Text Search” the advisor recommends KMP, Z-Function, Rabin-Karp and Suffix Array + LCP and shows the time and space complexity of each (Fig. 5.4).

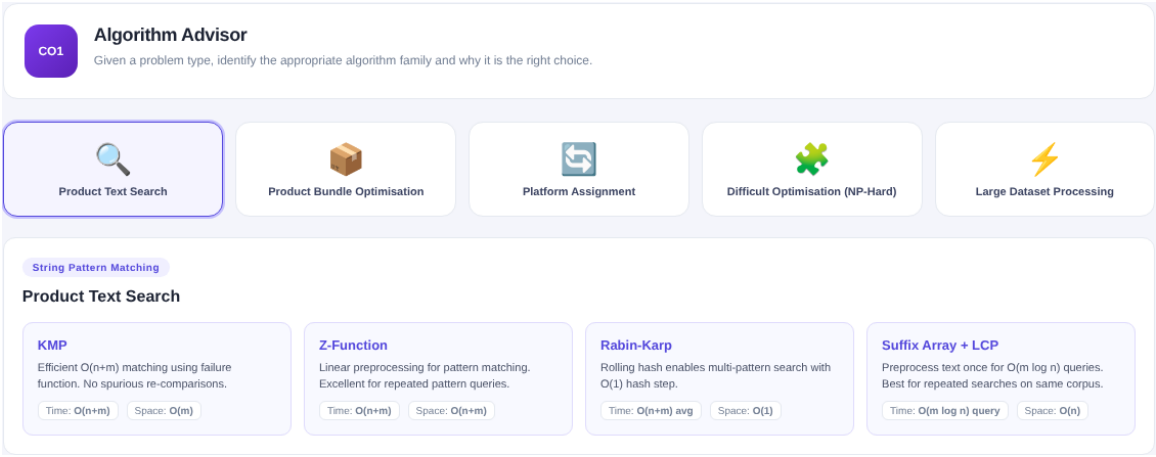


Fig. 5.4 Algorithm Advisor (CO1)

CO2 – Advanced search. For the query “wireless” all four algorithms find the same 4 products (Sony WH-1000XM5, JBL Tune 770NC, Logitech K380 Keyboard and boAt Airdopes 141) with 4 occurrences, as shown in Fig. 5.5. The measured times of one run were 0.24 ms (KMP), 0.63 ms (Z-Function), 0.13 ms (Rabin-Karp) and 8.31 ms (Suffix Array). Rabin-Karp was fastest in this run. The suffix array is slowest here because it is rebuilt for every query on the joined corpus; it pays off only when the same text is searched many times.

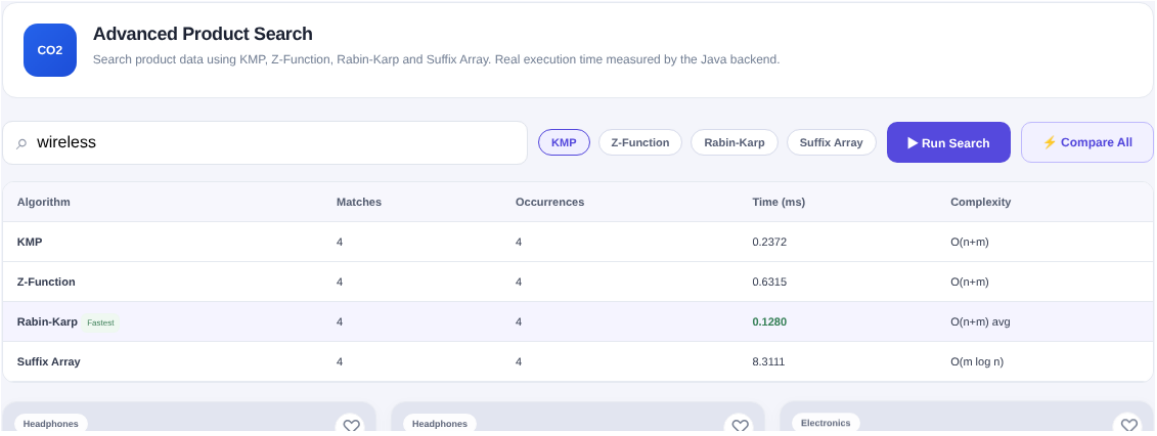


Fig. 5.5 Advanced search: KMP, Z-Function, Rabin-Karp and Suffix Array (CO2)

CO3 – Bundle optimiser. With a budget of ₹50,000 and the default product set, bitmask DP selects the combination with a total cost of ₹49,179 and a total score of 3,119 (sum of rating × 100), leaving ₹821 of the budget. The execution time was about 2 ms for 15 products (Fig. 5.6).

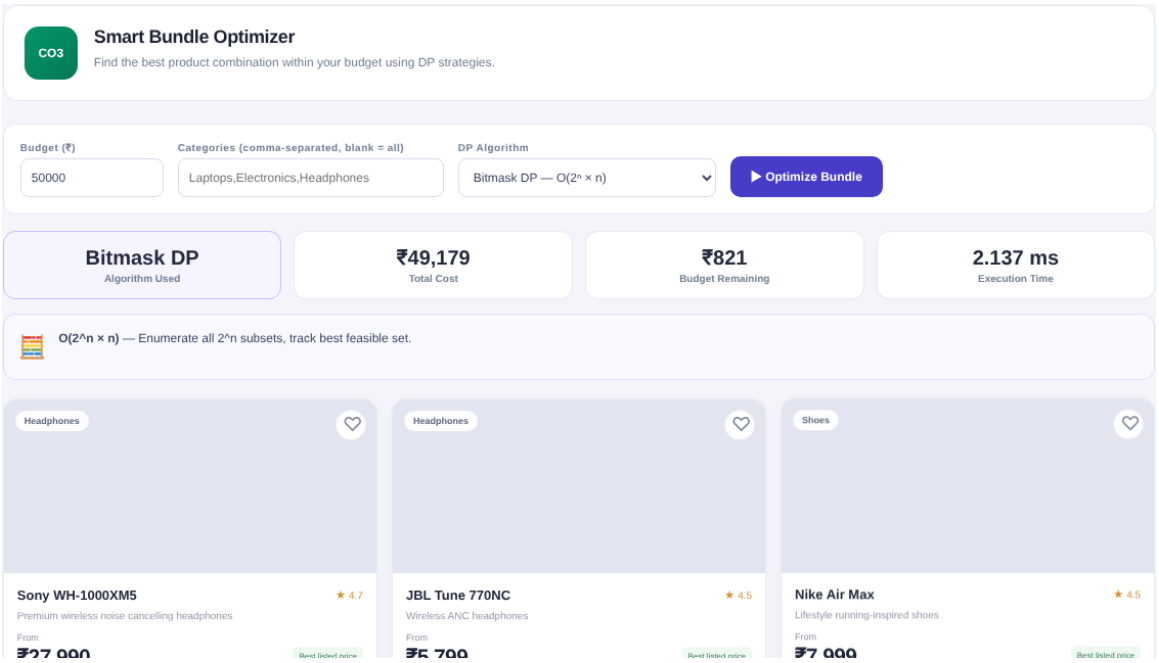
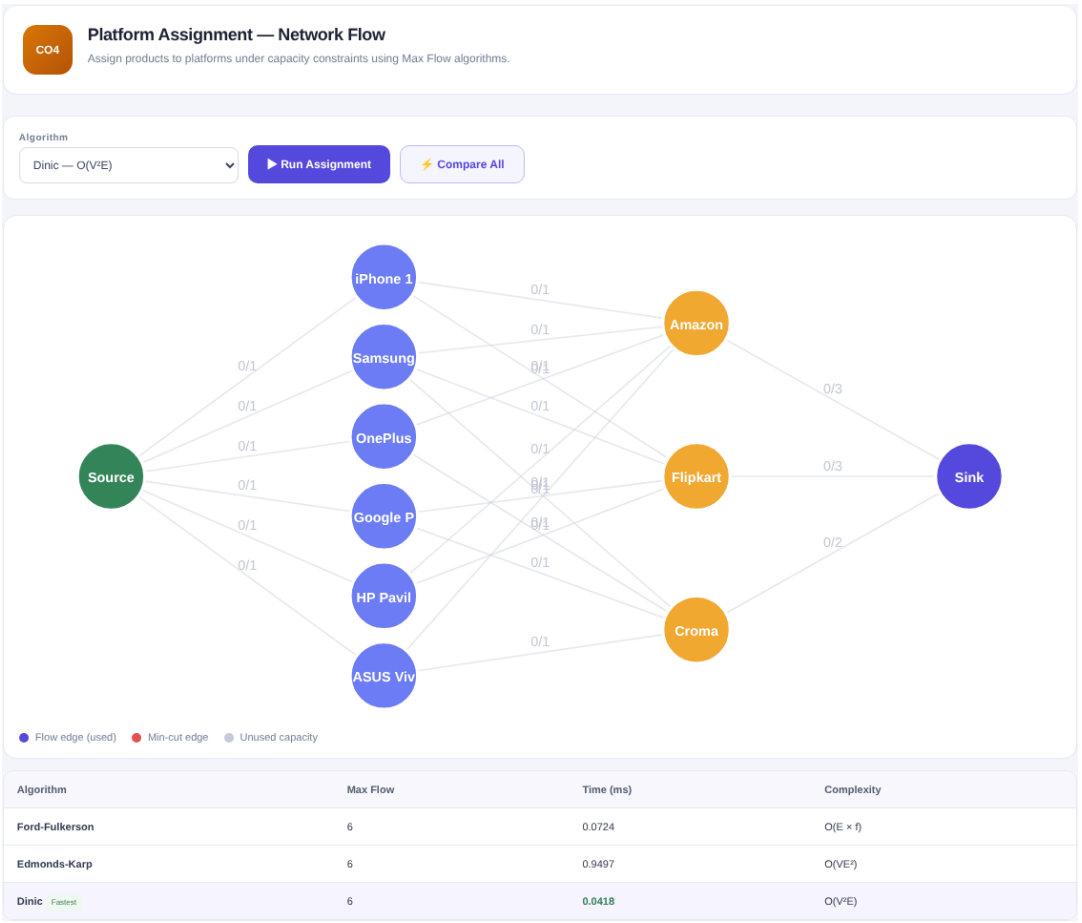


Fig. 5.6 Smart Bundle Optimizer using bitmask DP (CO3)

CO4 – Platform assignment. The flow network has 11 nodes (source, 6 products, 3 platforms and sink). Ford-Fulkerson, Edmonds-Karp and Dinic all return a maximum flow of 6, which means that all six products can be assigned to a platform within the capacity limits. The algorithms differ only in running time (in the run shown, 0.04 ms for Dinic, 0.07 ms for Ford-Fulkerson and 0.95 ms for Edmonds-Karp), as seen in Fig. 5.7.



Algorithm	Max Flow	Time (ms)	Complexity
Ford-Fulkerson	6	0.0724	O(E × f)
Edmonds-Karp	6	0.9497	O(VE²)
Dinic Fastest	6	0.0418	O(V²E)

Fig. 5.7 Platform assignment network flow (CO4)

CO5 – NP lab. For a budget of ₹30,000 and 10 products, the exact algorithm and the greedy approximation both select JBL Tune 770NC and Nike Air Max with a score of 900 and a cost of ₹13,798. The approximation ratio is therefore 1.00, while the exact algorithm needs $O(2^n)$ time and the greedy algorithm needs only O(n log n) (Fig. 5.8). The greedy method is not always optimal, but it scales to a large catalogue where the exact method cannot.

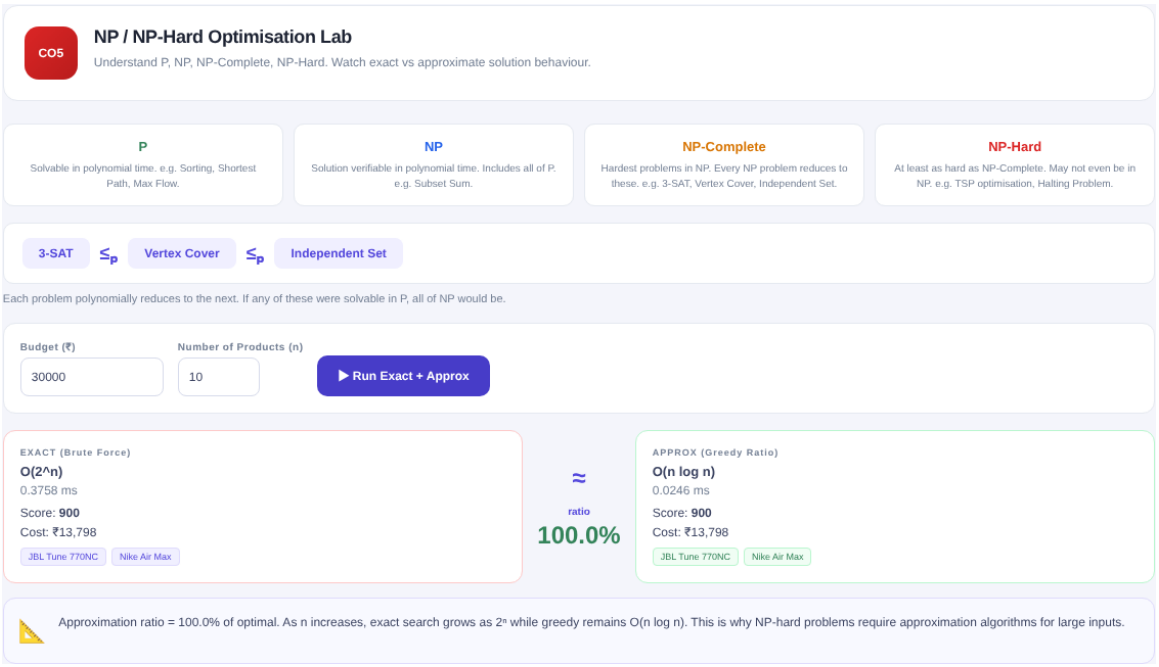


Fig. 5.8 NP / NP-Hard optimisation lab (CO5)

CO6 – Randomized algorithms. With seed 42 the Las Vegas QuickSelect runs on the 24 prices in about 0.03 ms, and the Monte Carlo Miller-Rabin test with 10 witnesses reports 999983 as probably prime (error probability at most 4^{-10}) in about 0.015 ms (Fig. 5.9).

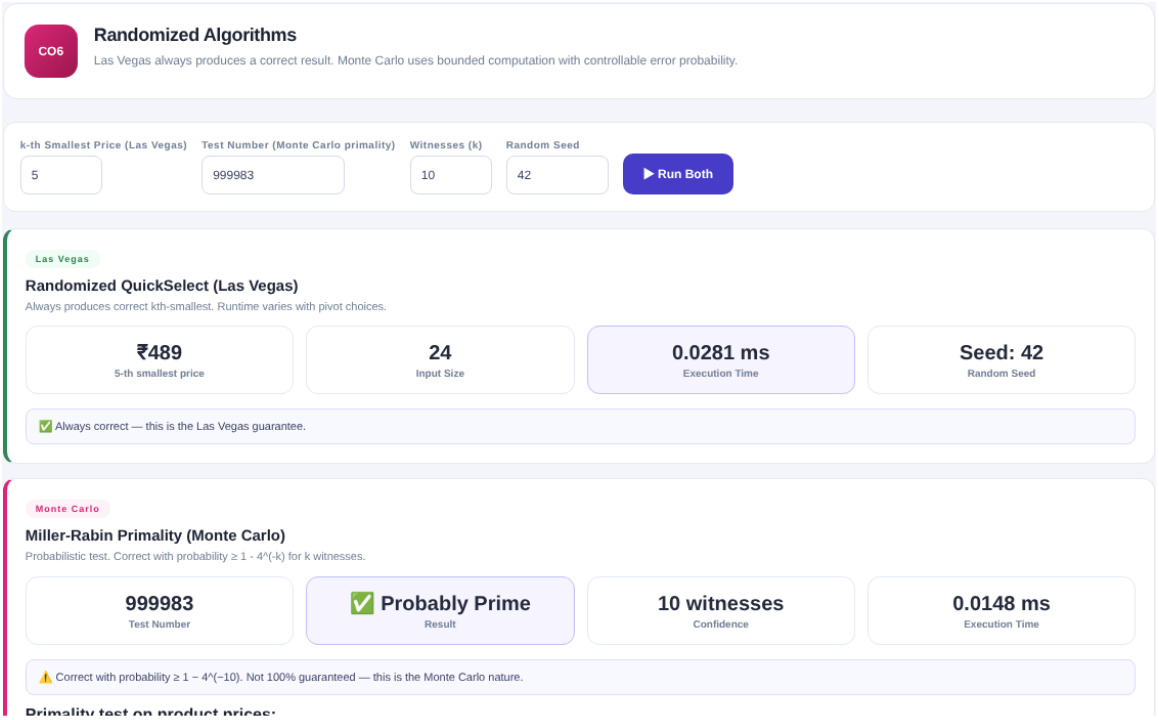


Fig. 5.9 Randomized algorithms: Las Vegas and Monte Carlo (CO6)

CO6 – Parallel algorithms. For the 24-element price array the work is 24, the span is 5 and the parallelism is 4.80. The parallel reduce gives the same total price (₹4,99,674) as the sequential

reduce. For such a small input the sequential version is faster (0.001 ms against about 0.46 ms for the reduce) because the thread-management overhead dominates, which the page reports honestly (Fig. 5.10).

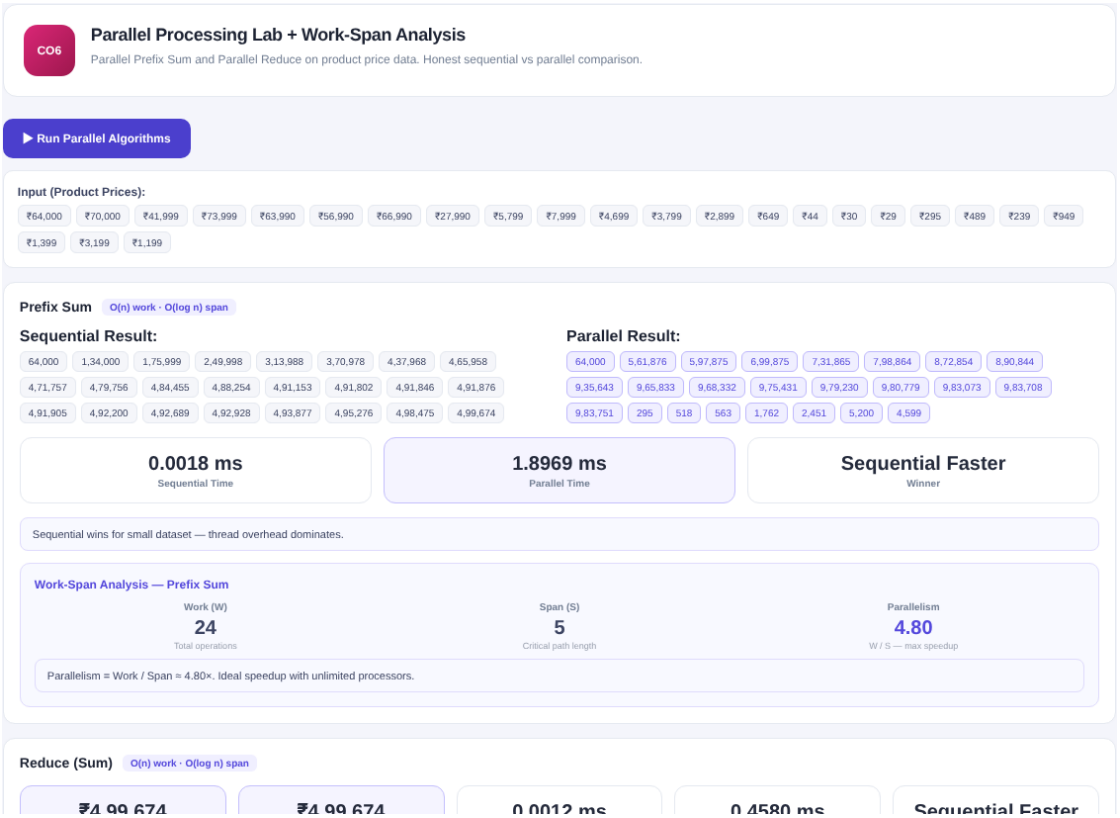


Fig. 5.10 Parallel processing lab and work-span analysis (CO6)

5.3 Performance observations

Table 5.1 Summary of measured results

Module	Input	Result	Time (one run)
KMP search	“wireless”, 24 products	4 products, 4 occurrences	0.24 ms
Z-Function search	“wireless”	4 products, 4 occurrences	0.63 ms
Rabin-Karp search	“wireless”	4 products, 4 occurrences	0.13 ms
Suffix Array search	“wireless”	4 products, 4 occurrences	8.31 ms
Bitmask DP bundle	Budget ₹50,000	Cost ₹49,179, score 3,119	≈ 2 ms
Dinic max flow	6 products, 3 platforms	Max flow = 6	0.04 ms
Exact vs greedy	Budget ₹30,000, n = 10	Both score 900, ratio 1.00	0.38 ms / 0.02 ms
Randomized QuickSelect	24 prices, seed 42	Always correct	0.03 ms
Parallel reduce	24 prices	Sum ₹4,99,674 (same as sequential)	0.46 ms (sequential 0.001 ms)

The results show that all pattern-matching algorithms return the same matches and differ only in running time and preprocessing cost; that the three max-flow algorithms agree on the flow value; and that the greedy approximation matched the exact optimum on this data. Times were measured on a normal laptop and vary slightly from run to run because the catalogue is very small.

6. CHALLENGES & LIMITATIONS

6.1 Issues faced during development

- Choosing suitable algorithms: many algorithms can solve a search or selection problem, and it was difficult to decide which one fits each part of the product-search problem.
- Exponential growth of the bitmask DP and brute-force methods, which become slow when more than about 20 products are selected.
- Modelling platform assignment as a flow network (source, products, platforms, sink) with correct capacities.
- Combining the front end and the Java server: handling JSON building by hand, cross-origin access and serving the static files from the built-in HTTP server.
- Measuring the running time fairly: for very small inputs the timing is affected by JVM warm-up and thread overhead, especially for the parallel algorithms.
- Keeping prices consistent: product prices differ between platforms and some platforms did not list every product.

6.2 Solutions applied

- A CO1 advisor module was added, which explains for each problem type why a particular algorithm family is the right choice.
- The number of products is capped at 20 for the bitmask DP and the exact NP method, and the polynomial-time knapsack DP and greedy approximation are provided as alternatives.
- The network was built with unit capacities between source, products and platforms, and three max-flow algorithms were implemented and cross-checked against the same flow value.
- A small JSON helper (quote, listJson, mapJson) and a common addCors() method were written, and the static-file handler was added to the root context.
- Each module reports its own measured time and the page honestly shows when the sequential version is faster than the parallel one.
- Only products and platforms with listed prices (Amazon and Flipkart) are shown in comparisons, and the page notes that the prices are reference examples.

6.3 Limitations of the System

- The catalogue is small (24 products) and the prices are stored in the program; the system

does not fetch live prices from the shopping websites.

- Spelling-mistake correction and auto-complete (Trie and Levenshtein distance), which were proposed in the project plan, are not yet implemented; the present search needs the typed text to be a substring of the product name or category.
- Product recommendation is based on category, price and rating sorting; it does not use user history or collaborative filtering.
- Subset DP, interval DP and tree DP in the bundle optimiser need further refinement: for example, the subset DP result for a budget of ₹50,000 selected a set whose cost (₹53,878) exceeded the budget, interval DP models the cost of comparing products rather than a purchase budget, and the tree DP option did not return a result in our test.
- In the parallel module the parallel prefix-sum output differs from the sequential output beyond the first block, so the block-combination step needs correction; the parallel reduce gives the correct sum. In the randomized module the rank k appears to be zero-based, so k = 5 returns the sixth smallest price.
- The server keeps all data in memory, has no authentication, and handles only GET requests.

7. CONCLUSION & FUTURE WORK

7.1 Conclusion

The Smart Product Search & Recommendation System shows how advanced data structures and algorithms can be applied to a real e-commerce problem. The system searches the catalogue with KMP, sorts the results by price, name and rating, compares prices across Amazon and Flipkart and, through the Advanced Algorithms Lab, applies string-matching, dynamic-programming, network-flow, approximation, randomized and parallel algorithms to the same data. The measured results confirm the expected behaviour: the pattern-matching algorithms agree on the matches, the max-flow algorithms agree on the flow, and the greedy approximation matches the optimum on the test data while being much cheaper to compute.

The project satisfies the six course outcomes (selection of algorithms, string matching, dynamic-programming-based recommendation, network flow, approximation techniques and randomized strategies) and helped the team understand the difference between theoretical complexity and practical running time. The known limitations listed in Section 6.3 give a clear direction for improvement.

7.2 Future Work

- Add a Trie for auto-complete suggestions and Levenshtein (edit) distance for automatic spelling correction, as planned in the project proposal.
- Replace the in-memory catalogue by a MySQL database accessed through JDBC and increase the number of products.
- Fetch live prices through the official APIs of the shopping platforms and add price-history graphs.
- Add personalised recommendations using item-based similarity and user history.
- Correct and complete the subset, interval and tree DP modules and the parallel prefix-sum, and add automated unit tests for every algorithm.
- Add user accounts, wish lists and price-drop alerts, and deploy the application on a cloud server.

7.3 Skills learned during development

- Designing and analysing algorithms: string matching, dynamic programming, network flow, approximation, randomized and parallel algorithms.
- Java programming with collections, recursion, priority queues, ForkJoinPool and the built-

in HTTP server.

- Building a REST API and a single-page web front end with HTML, CSS and JavaScript.
- Working as a team with Git and GitHub, dividing modules and integrating them.
- Testing, measuring and presenting the performance of algorithms, and writing a technical report.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] D. E. Knuth, J. H. Morris and V. R. Pratt, “Fast pattern matching in strings,” SIAM Journal on Computing, vol. 6, no. 2, pp. 323–350, 1977.

[3] R. M. Karp and M. O. Rabin, “Efficient randomized pattern-matching algorithms,” IBM Journal of Research and Development, vol. 31, no. 2, pp. 249–260, 1987.

[4] U. Manber and G. Myers, “Suffix arrays: a new method for on-line string searches,” SIAM Journal on Computing, vol. 22, no. 5, pp. 935–948, 1993.

[5] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” Canadian Journal of Mathematics, vol. 8, pp. 399–404, 1956.

[6] J. Edmonds and R. M. Karp, “Theoretical improvements in algorithmic efficiency for network flow problems,” Journal of the ACM, vol. 19, no. 2, pp. 248–264, 1972.

[7] E. A. Dinic, “Algorithm for solution of a problem of maximum flow in networks with power estimation,” Soviet Mathematics Doklady, vol. 11, pp. 1277–1280, 1970.

[8] M. O. Rabin, “Probabilistic algorithm for testing primality,” Journal of Number Theory, vol. 12, no. 1, pp. 128–138, 1980.

[9] J. Kleinberg and É. Tardos, Algorithm Design. Boston, MA, USA: Pearson/Addison-Wesley, 2006.

[10] Oracle, “Java Platform, Standard Edition API Specification: com.sun.net.httpserver,” https://docs.oracle.com/en/java/javase/21/docs/api/jdk.httpserver/com/sun/net/httpserver/package-summary.html

[11] Mozilla Developer Network, “Fetch API,” https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[12] Project source code: KLH-CSE-2026-2027-TEAM4-SmartProductSearch (GitHub repository), Department of CSE, KLEF.